Doc-QA

Document Question Answering System

Django 4.2 | LangChain | Docker | RAG Pipeline | Hybrid Search

Developed by Alireza Rezaei

RAG · BM25 · Semantic Chunking · RRF Fusion · OpenRouter LLM

What the Project Asked For

Mandatory Requirements

- Django as web framework
- Docker for deployment
- LangChain for LLM integration
- Full usable REST API
- Django Admin as UI (no separate frontend)

Expected Capabilities

- Add, edit & delete documents
- Support .docx file format
- Store full document text
- Submit questions
- Find relevant content
- Advanced document retrieval
- Generate text answers
- Store Q&A history

System Architecture

PHASE 1 — Indexing

Happens once on upload

1. Upload .docx file
2. Extract text + heading structure
3. Semantic & structure-aware chunking
4. Embed chunks → 384-dim vectors
5. Compute BM25 term frequencies
6. Persist to SQLite database

PHASE 2 — Querying

Happens on every question

1. Receive question
2. Embed question → vector
3. Dense search (cosine similarity)
4. Sparse search (BM25 scoring)
5. Fuse rankings with RRF
6. Send top-4 chunks + question to LLM
7. Return answer + source attribution

Project Structure

documents/ — Ingestion App

- models.py — Document + DocumentChunk (stores text, embeddings, BM25)
- processing.py — Full pipeline: extract → chunk → embed → save
- signals.py — Triggers pipeline automatically on upload
- views.py — Full CRUD API (ModelViewSet)

qa/ — Query App

- pipeline.py — Hybrid retrieval + LLM generation
- models.py — QAHistory (persists every Q&A + source docs)
- admin.py — Ask a Question page in Django Admin
- views.py — POST /api/ask/ · GET /api/history/

core/
Django config, settings, URLs

templates/
Django Admin custom pages

Dockerfile + docker-compose
One-command deployment

Smart Chunking — From Naive to Semantic

✕ Before — Naive Fixed-Size

- Split every 500 words — no matter what
- Can break mid-sentence or mid-topic
- Mixes two unrelated subjects in one chunk
- Ignores document headings entirely
- Poor retrieval quality as a result

✓ After — Semantic + Structural

- ✓ **Structural signal:** Heading detected → always split
- ✓ **Semantic signal:** Cosine similarity drops below 0.45 → split
- ✓ **Size safety net:** Chunk exceeds 600 words → force split
- ✓ **Result:** Each chunk = one coherent topic
- ✓ **Benefit:** Better retrieval, more accurate answers

Hybrid Search — Why Two Methods Beat One

Dense Search

Semantic Similarity

Strength

Finds paraphrases & related ideas even with different words

Weakness

Misses exact keywords — article numbers, names, codes

```
score = cos(question_vec, chunk_vec)
```

BM25

Keyword Scoring

Strength

Never misses an exact keyword match — names, codes, terms

Weakness

Fails when user paraphrases — zero score if no word overlap

```
score = Σ IDF(t) × TF_norm(t, chunk)
```

RRF Fusion

Combined Ranking

Strength

Best of both — chunks that rank high in BOTH methods win

Weakness

Scale-free: works regardless of score magnitude differences

```
RRF(d) = 1/(60+rank_dense) + 1/(60+rank_bm25)
```

Database Schema & API Endpoints

Database Models

Document

- id
- title
- file
- extracted_text
- uploaded_at

DocumentChunk

- id
- document (FK)
- content
- embedding_json
- bm25_tf_json
- chunk_index

QAHistory

API Endpoints

POST `/api/documents/`

Upload document

GET `/api/documents/`

List documents

GET `/api/documents/{id}/`

Get document

PUT `/api/documents/{id}/`

Update document

DELETE `/api/documents/{id}/`

Delete document

POST `/api/ask/`

Ask a question

GET `/api/history/`

View Q&A history

Docker — One Command to Run

```
$ docker compose up --build
```

Dockerfile

- Base: python:3.11-slim
- Installs all dependencies
- Pre-downloads MiniLM model (~90MB) at build time
- Creates media/ and data/ directories
- Runs migrate then starts server

docker-compose.yml

- Defines the web service
- Maps port 8000 → 8000
- Passes .env variables into container
- sqlite_data volume → DB survives restarts
- media_data volume → uploads survive restarts

.dockerignore

- Excludes venv/ (huge, not needed)
- Excludes .env (secrets stay local)
- Excludes db.sqlite3 (container has its own)
- Excludes __pycache__ and .git
- Keeps image small and clean

What I have Built

Full RAG Pipeline

Two-phase: index on upload, retrieve on query

Django Admin UI

Ask questions directly from the admin panel

Semantic Chunking

Structure-aware + cosine similarity between paragraphs

REST API

7 endpoints — full document lifecycle + Q&A + history

Hybrid Search

Dense embeddings + BM25 + RRF fusion

Docker

One command to build and run the entire system

Thank you